Reporte

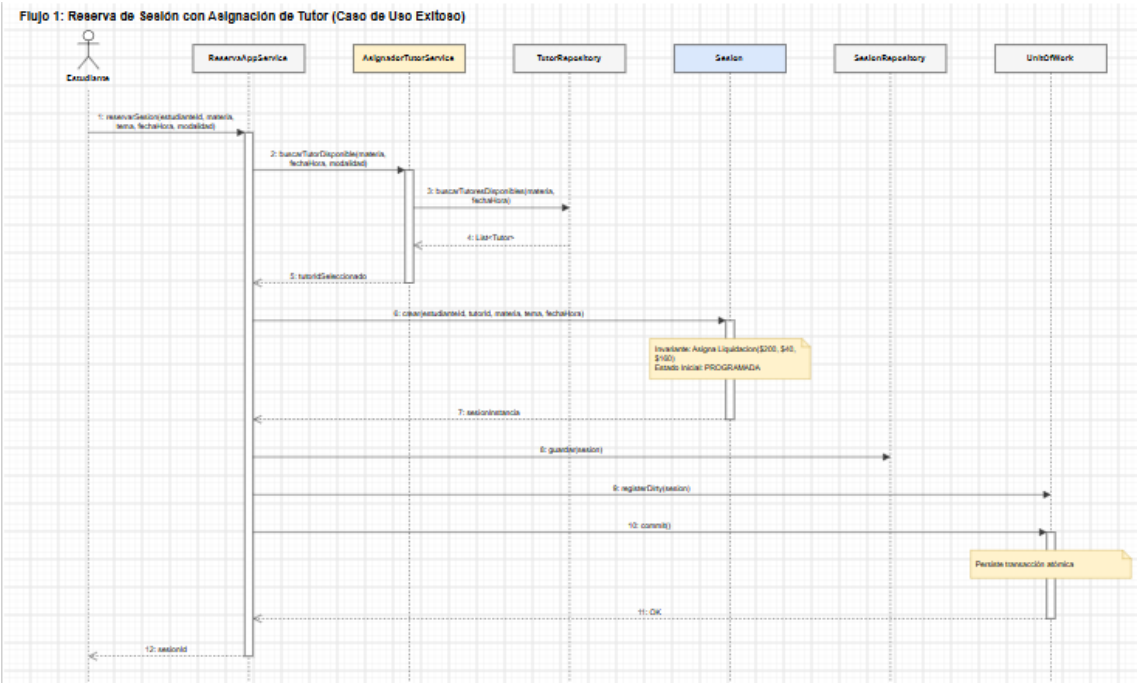
Número del equipo: 8	
Nombres: Esther Garza Cano Luis Ángel David Cavazos Reyes Emilio Garza Arizpe	**Matrícula**: 2143077 2142838 2143702
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Actividad: Diagramas UML AulaPlus	
Fecha: 17/10/2026	
Bibliografia (Formato APA)	

4. Diagramas UML

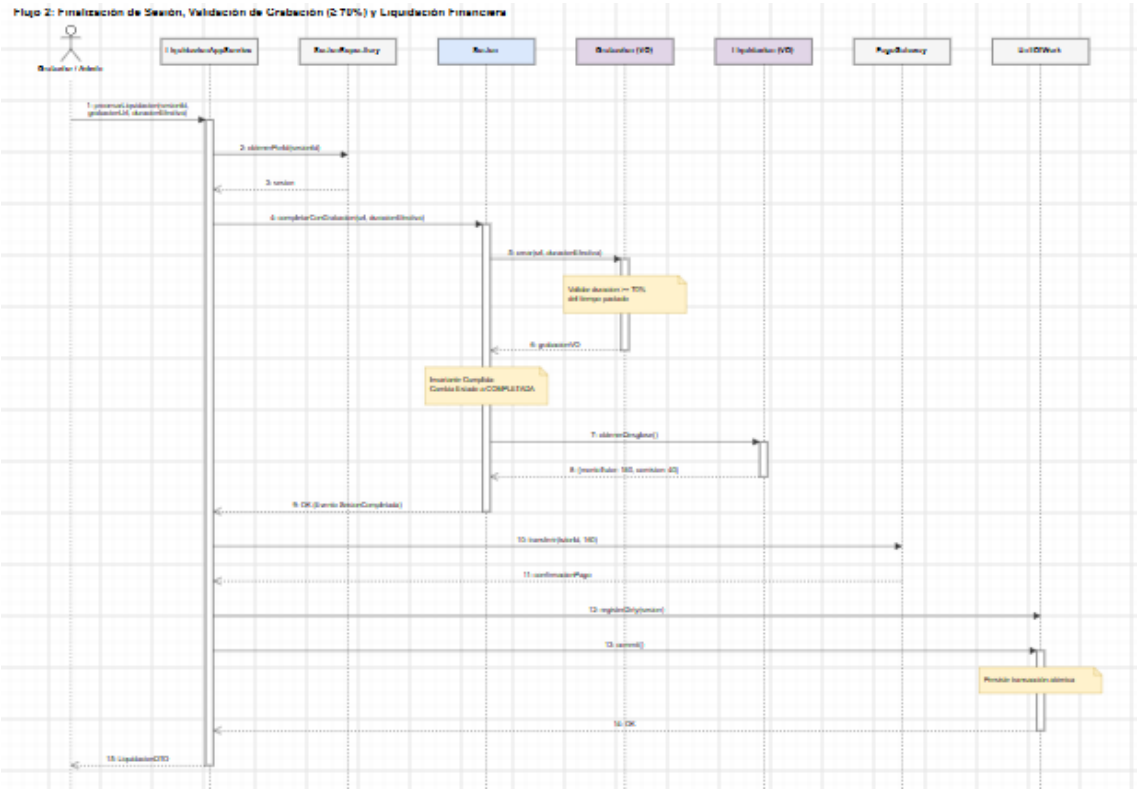
4.1 Diagramas de dominio

4.2 Diagramas de secuencia

4.2.0 Flujo 1: Reserva de Sesión con Asignación de Tutor (Caso de Uso Exitoso)



4.2.1Flujo 2: Finalización de Sesión, Validación de Grabación (≥ 70%) y Liquidación Financial



4.3 Registros de ADR

ADR-001: Selección del Patrón Domain Model sobre Transaction Script

Atributo	Detalle
Estatus	**Aceptado**
Contexto	El sistema *Aula Plus* debe gestionar reglas de negocio complejas como la validación de cancelación (≥ 48h), la verificación de duración de clase (≥ 70%) y la distribución inmutable de tarifa ($200 / $40 / $160). Un enfoque centrado en base de datos provocaría dispersión de lógica.
Decisión	Adoptar el patrón de arquitectura de lógica de negocio **Domain Model (Modelo de Dominio)** complementado con Domain-Driven Design (DDD)
Consecuencias Positivas	• Alta encapsulación de reglas e invariantes dentro de entidades y objetos de valor.

	• Facilidad de pruebas unitarias al no depender de la base de datos. • Alineación directa con el Lenguaje Ubicuo del negocio.
Consecuencias Negativas	• Mayor curva de aprendizaje y complejidad inicial en el mapeo de persistencia (ORM).

ADR-002: Descoplamiento de Lectura y Escritura mediante CQRS Básico

Atributo	Detalle
Estatus	**Aceptado**
Contexto	Las operaciones de consulta (ej. historial de tutorías o catálogo de tutores) requieren alta velocidad y DTOs planos, mientras que las operaciones de modificación demandan cargar agregados pesados para validar invariantes.
Decisión	Implementar **CQRS Básico (Command Query Responsibility Segregation)**, separando los comandos (*Commands*) que pasan por la capa de aplicación y el modelo de dominio, de las consultas (*Queries*) que leen directamente proyecciones optimizadas DTO.
Consecuencias Positivas	• Optimización de rendimiento en consultas sin sobrecargar el modelo de dominio. • Código más limpio y alineado al principio SOLID de Responsabilidad Única (SRP).
Consecuencias Negativas	• Duplicidad en la definición de estructuras de datos (DTOs de lectura vs. Entidades de escritura).

ADR-003: Gestión Transaccional mediante el Patrón Unit of Work

Atributo	Detalle
Estatus	**Aceptado**
Contexto	Casos de uso como notificarAusencia o procesarLiquidacion afectan a múltiples agregados (Tutor y Sesion) en una misma operación, requiriendo garantías ACID.
Decisión	Utilizar el patrón **Unit of Work (Unidad de Trabajo)** en la Capa de Servicios de Aplicación para rastrear cambios en memoria y realizar commits/rollbacks atómicos.
Consecuencias Positivas	• Garantiza consistencia transaccional sin acoplar la capa de aplicación a una tecnología SQL/NoSQL específica. • Minimiza el número de llamadas a la base de datos agrupando escrituras.
Consecuencias Negativas	• Requiere implementar un mecanismo explícito de rastreo de objetos (*Change Tracking*) en memoria.

4.4 Matriz de trazabilidad actualizada

ID requerimiento	Regla de negocio	Componente del dominio / patrón	Método / Módulo Responsable	Artefacto / Diagrama donde se evidencia
BRD-REQ-01	Búsqueda y emparejamiento de tutor por materia, horario y zona.	*Domain Service:* AsignadorTutorService.	buscarTutorDisponible()	Diagrama de Secuencia 1 & Capa de Servicios.
BRD-REQ-02	Regla de Cancelación con reembolso automático si es ≥ 48h	*Entity:* SolicitudCancelacion dentro del Agregado Sesion.	sesion.solicitarCancelacion()	Ficha de Agregado Sesion & ADR-001.

BRD-REQ-03	Evidencia por Grabación con duración efectiva ≥ 70% de la clase.	*Value Object:* Grabacion en Agregado Sesion.	sesion.completarConGrabacion()	Diagrama de Secuencia 2 & ADR-001.
BRD-REQ-04	Desglose Financiero Inmutable ($200 Total = $40 Comisión + $160 Tutor).	*Value Object:* Liquidacion.	liquidacion.obtenerDesglose()	Diagrama de Secuencia 2 & Ficha de Agregado Sesion.
BRD-REQ-05	Límite estricto de máximo 2 calificaciones por sesión (0 … 2).	*Entity: Calificacion dentro de Agregado Sesion.*	sesion.agregarCalificacion()	Ficha de Agregado Sesion & Principio SOLID (S).
BRD-REQ-06	Consistencia transaccional multiobjeto al notificar ausencias/liquidaciones.	*Pattern:* UnitOfWork	UnitOfWork.commit()	ADR-003 & Tabla de Application Services.

5.4 Bitácora del proyecto

Entregable	Consulta a la IA	Respuesta de la IA	Posibles Errores	Decisión final
Etapa 1: Casos de Uso y Selección de Patrón	"¿Cómo debemos estructurar la tabla de Casos de Uso y qué patrón arquitectónico de lógica de negocio es el más apto según la complejidad del BRD de Aula Plus?"	Sugirió identificar los 5 flujos centrales (Reserva, Disponibilidad, Cancelación, Liquidación y Ausencia) y presentó una matriz comparativa entre *Transaction Script*, *Table Module* y *Domain Model*	• Riesgo de colocar la lógica de cobro y validación de horarios dentro de scripts simples (*Transaction Script*), dispersando las reglas de negocio en la base de datos	**Decisión:** Seleccionar **Domain Model (DDD)**. Se formalizaron los 5 Casos de Uso (CU-01 al CU-05) integrando explícitamente las reglas críticas de la plataforma (monto inmutable de $200 MXN, regla de ≥ 48h y duración de grabación ≥ 70%).
Etapa 1.1: Invariantes y Encapsulamiento	"¿En qué capa debe validarse la regla de no solapamiento de horarios entre las tutorías y la disponibilidad del tutor?"	Recomendó que el *Application Service* coordine la consulta, pero que la validación lógica viva dentro de la entidad Tutor y el *Domain Service* AsignadorTutorService.	• Error común: Delegar la validación de solapamiento a un *stored procedure* o un script SQL en lugar de resolverlo en el Dominio	**Decisión:** Mantener la validación del cruce de franjas horarias estrictamente encapsulada en el método tutor.registrarDisponibilidad() y el *Domain Service* de asignación.
Etapa 2: Modelo y Mapa de Dominio	"¿De qué manera debemos delimitar los Agregados, Entidades y Value Objects para no sobrecargar el Agregado central Sesión?"	Propuso estructurar el dominio en 3 Agregados principales (Estudiante, Tutor y Sesion), definiendo Grabacion y Liquidacion como *Value Objects* inmutables.	• Dudas sobre si SolicitudCancelacion y Calificacion debían ser Agregados independientes o vivir dentro de Sesion. • Supuesto inicial de manejar la comisión bancaria de manera variable.	**Decisión:** Mantener Sesion como el **Agregado Central Root**, conteniendo a SolicitudCancelacion y a Calificacion (con límite de 0 … 2). Definir Liquidacion como VO inmutable con desglose estricto ($200 MXN = $40 comisión + $160 tutor) y desacoplar la asignación mediante el *Domain Service* AsignadorTutorService.
Etapa 2.1: Gestión de Excepciones del Dominio	"¿Cómo debemos manejar los fallos en las reglas de negocio (ej. si la grabación dura menos del 70%)	Explicó que el Agregado Sesion debe lanzar excepciones explícitas de Dominio (ej. DuracionInsuficienteException) sin capturar	• Supuesto erróneo: Devolver valores boolean o códigos de error numéricos en lugar de Excepciones de	**Decisión:** Modelar Excepciones de Dominio personalizadas. El *Application Service* las captura para hacer UnitOfWork.rollback() y

	desde el Dominio hacia las capas superiores?"	errores técnicos de base de datos.	Dominio fuertemente tipadas.	retornar la respuesta adecuada al cliente.
Etapa 3: Capa de Servicios y Arquitectura	"¿Cuál es la responsabilidad exacta de un Application Service y cómo debemos estructurar los repositorios y la persistencia transaccional?"	Definió que los *Application Services* solo orquestan casos de uso sin ejecutar reglas de negocio. Propuso utilizar contratos de repositorios en Lenguaje Ubicuo, el patrón *Unit of Work* y una separación básica de CQRS.	• Peligro de acoplar la capa de servicios a consultas SQL o de escribir lógica de validación de tiempo dentro del servicio y no en el dominio.	**Decisión:** Implementar 4 *Application Services* (ReservaAppService, TutorGestionAppService, EvaluacionAppService, LiquidacionAppService). Toda modificación pasa por *Commands* con UnitOfWork.commit(), mientras las consultas van por *Queries* (CQRS). Se reestructuró la tabla para incluir las **6 columnas exactas** requeridas en la plantilla
Etapa 3.1: Definición de DTOs en CQRS	"¿Cómo evitamos la fuga de entidades de dominio hacia la vista al consultar catálogos o historiales de tutorías?"	Propuso la creación de DTOs planos e inmutables para el lado de lectura (*Queries*), desconectados del ciclo de vida de los Agregados de escritura.	• Riesgo de exponer métodos mutables del Agregado Sesion en la API Web si se reusan las entidades de dominio como respuesta JSON.	**Decisión:** Adoptar DTOs de lectura directos (HistorialEstudianteDTO, CatalogoTutorDTO) en la capa de *Queries*, garantizando la separación de responsabilidades propuesta en CQRS
Etapa 3.2: Paginación y Criterios en Repositorios	"¿Cómo definir la firma del contrato del repositorio cuando se buscan tutores disponibles para evitar sobrecarga de memoria?"	Sugirió incluir parámetros filtrados (materia, fechaHora, zona) en la interfaz de TutorRepository expresados totalmente en lenguaje de negocio.	• Error común: Definir métodos de repositorio genéricos tipo findAll() y filtrar las listas en memoria dentro de la aplicación.	**Decisión:** Formalizar la firma buscarTutoresDisponibles(materia, fechaHora, zona) en TutorRepository para delegar la eficiencia del filtro a la infraestructura sin perder la abstracción del negocio.
Etapa 4: Artefactos UML, ADRs y Estructura	"¿Cómo modelar los diagramas de secuencia para los flujos complejos y qué registros ADR respaldan las decisiones del	Generó el código Mermaid para los diagramas de secuencia de Reserva y Liquidación, redactó 3 registros ADR (Domain Model, CQRS y Unit of	• Riesgo de no reflejar la validación de la grabación (≥ 70%) o la transferencia externa al pasaje de pago dentro de	**Decisión:** Aprobar los **Diagramas de Secuencia para el Flujo 1 y Flujo 2**, ratificar los **3 ADRs** formalizados y consolidar la **Matriz de Trazabilidad de Requerimientos** (BRD-REQ-

	diseño de software?"	Work) y propuso el layout de carpetas para GitHub.	los diagramas de secuencia. • Faltante de una matriz que garantizara trazabilidad directa con el BRD.	01 a BRD-REQ-06). Se estableció la estructura modular src/domain, src/application, src/infrastructure para la entrega en el repositorio de GitHub.
Etapa 4.1: Estrategia de Pruebas Unitarias y SOLID	"¿Cómo facilitamos la creación de pruebas unitarias sobre las reglas comerciales sin depender de bases de datos o pasarelas reales?"	Demostró cómo la aplicación de SOLID (Inversión de Dependencias - DIP) permite utilizar Mocks o Repositorios En Memoria para probar los Agregados y Servicios de Aplicación.	• Riesgo de requerir una instancia levantada de base de datos SQL para probar si la regla de ≥ 48h funciona correctamente.	**Decisión:** Diseñar la arquitectura desacoplada mediante interfaces de repositorios (SesionRepository), haciendo posible probar el 100% de la lógica de negocio mediante *Unit Tests* aislados y rápidos.